

Leetcode 239. Sliding Window Maximum

Achyutha Duddebanda

June 9th 2026

Problem

We are given an array and we have to return another array of sliding window maximums. A sliding window maximum contains the largest element present in the window. Here is a simple example illustrating this idea:

Window position		Max
[10 3 4]	-3 5 -4 -13 8	10
10 [3 4 -3]	5 -4 -13 8	4
10 3 [4 -3 5]	-4 -13 8	5
10 3 4 [-3 5 -4]	-13 8	5
10 3 4 -3 [5 -4 -13]	8	5
10 3 4 -3 5 [-4 -13 8]		8

Solution

To solve this question, we will make use of the **deque** (double-ended queue) data structure. This means that we can add and remove items from the front and back of the queue in $\mathcal{O}(1)$ time. In order to easily access the largest value in the queue, we need to ensure that the maximum value of the sliding window is always at the head of the queue. Also, if an element is popped out of the queue, then the head of the queue must contain the largest element within the new window. In order to achieve this functionality, we have to ensure that the queue is in decreasing order at all times, i.e., a **monotonic decreasing** queue. To do this, we will perform the following steps:

1. If the largest element leaves the window (`nums[l - 1] == q.peak()`), remove it from the head of the queue.
2. Add the new element (`nums[r]`) to the queue while still maintaining decreasing order of the queue.

Note: in order to maintain decreasing order in the queue while still adding `nums[r]`, all elements that are less than `nums[r]` need to be removed from the end of the queue until the last element of the queue is larger than `nums[r]`.

Example

Now, let's see what the deque looks like at each step so that we can visualize the result:

Window position	Max	Deque
[10 3 4] -3 5 -4 -13 8	10	[10, 4]
10 [3 4 -3] 5 -4 -13 8	4	[4, -3]
10 3 [4 -3 5] -4 -13 8	5	[5]
10 3 4 [-3 5 -4] -13 8	5	[5, -4]
10 3 4 -3 [5 -4 -13] 8	5	[5, -4, -13]
10 3 4 -3 5 [-4 -13 8]	8	[8]

Note that this algorithm works because we are only interested in the largest value for each window. Therefore, it's acceptable for us to discard unnecessary information, which is why we can remove all elements that are less than `nums[r]`.

Time and Space Complexity

- Time complexity: $\mathcal{O}(n)$
- Space complexity: $\mathcal{O}(k)$

where n is `nums.length` and k is the length of the deque. The time complexity is linear despite needing a `while` loop to maintain order since no one element in `nums` can be deleted or inserted more than once. Even for a strictly increasing array like `[0, 1, 2, 3, 4, 5]`, only one element needs to get removed every iteration since the queue is built sequentially.